

 c0a25046bf / ProjExD_Group14

 | 

 **Code**  Issues  Pull requests **1**  Agents  Actions  Projects  Wiki  Security and quality  Insights

 On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#). 

  C0A25046/ステ...  **ProjExD_Group14** / koukaton_up.py 

 Go to file  T 

 **c0a25046bf** ギミックを1層1つに変更 7e23673 · 47 minutes ago 

331 lines (258 loc) · 11.5 KB

Code **Blame**

 **Raw**   

```
1  import pygame
2  import sys
3  import os
4  import random
5
6  # カレントディレクトリの固定
7  os.chdir(os.path.dirname(os.path.abspath(__file__)))
8
9  WIDTH, HEIGHT = 600, 800
10 FPS = 60
11 GRAVITY = 0.6
12
13 BLACK = (20, 20, 25)
14 BLUE = (100, 150, 255)
15 WHITE = (255, 255, 255)
16 GOLD = (255, 215, 0)
17
18 # ギミック床の色定義を復活
19 COLOR_NORMAL = (120, 120, 120)    # 緑 (通常)
20 COLOR_FAKE = (180, 0, 180)        # 紫 (すり抜ける)
21 COLOR_ICE = (100, 200, 255)       # 水色 (滑る)
22 COLOR_TRAMPOLINE = (255, 140, 0)  # オレンジ (大ジャンプ)
23 COLOR_TRAP = (255, 50, 50)        # 赤 (最初に戻る)
24 COLOR_RED_UI = (255, 100, 100)    # UI用赤
25
26 # 種類から色を取得する辞書
27 COLOR_MAP = {
28     "normal": COLOR_NORMAL,
29     "fake": COLOR_FAKE,
30     "ice": COLOR_ICE,
31     "trampoline": COLOR_TRAMPOLINE,
32     "trap": COLOR_TRAP
33 }
34
35 # =====
36 # 1. データ構造 (クラス) の定義
37 # =====
38
39 class Player:
40     def __init__(self):
41         self._image = pygame.Surface((30, 40))
42         self._image.fill(BLUE)
43         self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT - 100))
44
45         self._vel_x = 0
46         self._vel_y = 0
47         self._on_ground = False
48
```

```

49         self._is_charging = False
50         self._charge_power = 0
51         self._max_charge = 20.0
52         self._direction = 1
53
54         self._is_clear = False
55         self._on_ice = False
56         self._is_reset = False
57
58         # --- ゲッター ---
59         def get_image(self): return self._image
60         def get_rect(self): return self._rect
61         def get_vel_x(self): return self._vel_x
62         def get_vel_y(self): return self._vel_y
63         def get_on_ground(self): return self._on_ground
64         def get_is_charging(self): return self._is_charging
65         def get_charge_power(self): return self._charge_power
66         def get_max_charge(self): return self._max_charge
67         def get_direction(self): return self._direction
68         def get_is_clear(self): return self._is_clear
69         def get_on_ice(self): return self._on_ice
70         def get_is_reset(self): return self._is_reset
71
72         # --- セッター ---
73         def set_vel_x(self, value): self._vel_x = value
74         def set_vel_y(self, value): self._vel_y = value
75         def set_on_ground(self, value): self._on_ground = value
76         def set_is_charging(self, value): self._is_charging = value
77         def set_charge_power(self, value): self._charge_power = value
78         def set_direction(self, value): self._direction = value
79         def set_is_clear(self, value): self._is_clear = value
80         def set_on_ice(self, value): self._on_ice = value
81         def set_is_reset(self, value): self._is_reset = value
82
83
84     class Platform:
85     def __init__(self, x, y, w, h, color, plat_type="normal"):
86         self._image = pygame.Surface((w, h))
87         self._image.fill(color)
88         self._rect = self._image.get_rect(topleft=(x, y))
89         self._type = plat_type
90
91         def get_image(self): return self._image
92         def get_rect(self): return self._rect
93         def get_type(self): return self._type
94
95
96     # =====
97     # 2. クラスを操作する関数
98     # =====
99
100     def update_player(player, keys, platforms, goal_block):
101         if not player.get_is_clear():
102             if player.get_on_ground():
103
104                 if player.get_on_ice():
105                     player.set_vel_x(player.get_vel_x() * 0.96)
106                 else:
107                     player.set_vel_x(0)
108
109                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
110                     player.set_direction(-1)
111                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
112                     player.set_direction(1)
113
114                 if keys[pygame.K_SPACE]:
115                     player.set_is_charging(True)

```

```
116         current_power = player.get_charge_power()
117         if current_power < player.get_max_charge():
118             player.set_charge_power(current_power + 0.4)
119     else:
120         if player.get_is_charging():
121             power = player.get_charge_power()
122             direction = player.get_direction()
123
124             player.set_vel_y(-power * 0.9 - 5)
125             player.set_vel_x(direction * (power * 0.4 + 2))
126
127             player.set_is_charging(False)
128             player.set_charge_power(0)
129             player.set_on_ground(False)
130             player.set_on_ice(False)
131
132     if not player.get_on_ground():
133         player.set_vel_y(player.get_vel_y() + GRAVITY)
134
135     rect = player.get_rect()
136
137     rect.x += player.get_vel_x()
138     if rect.left < 0:
139         rect.left = 0
140         player.set_vel_x(player.get_vel_x() * -0.5)
141         player.set_direction(1)
142     if rect.right > WIDTH:
143         rect.right = WIDTH
144         player.set_vel_x(player.get_vel_x() * -0.5)
145         player.set_direction(-1)
146
147     rect.y += int(player.get_vel_y())
148
149     goal_rect = goal_block.get_rect()
150     if rect.collidirect(goal_rect):
151         if player.get_vel_y() < 0:
152             rect.top = goal_rect.bottom
153             player.set_vel_y(0)
154             player.set_is_clear(True)
155
156     player.set_on_ground(False)
157     player.set_on_ice(False)
158
159     if player.get_vel_y() > 0:
160         for plat in platforms:
161             if plat.get_type() == "fake":
162                 continue
163
164             plat_rect = plat.get_rect()
165             if rect.collidirect(plat_rect):
166                 if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
167
168                     if plat.get_type() == "trampoline":
169                         rect.bottom = plat_rect.top
170                         player.set_vel_y(-25)
171                         player.set_on_ground(False)
172                         break
173
174                     if plat.get_type() == "trap":
175                         rect.center = (WIDTH // 2, HEIGHT - 100)
176                         player.set_vel_y(0)
177                         player.set_vel_x(0)
178                         player.set_is_reset(True)
179                         break
180
181                 rect.bottom = plat_rect.top
182                 player.set_vel_y(0)
```

```

183         player.set_on_ground(True)
184
185         if plat.get_type() == "ice":
186             player.set_on_ice(True)
187         else:
188             player.set_vel_x(0)
189         break
190
191
192 # =====
193 # 3. メインループ
194 # =====
195
196 ❷ def main():
197     pygame.init()
198     screen = pygame.display.set_mode((WIDTH, HEIGHT))
199     pygame.display.set_caption("Jump King - Colored Gimmicks")
200     clock = pygame.time.Clock()
201     font = pygame.font.SysFont(None, 36)
202     large_font = pygame.font.SysFont(None, 72)
203
204     player = Player()
205     platforms = []
206
207     platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, COLOR_NORMAL, "normal"))
208
209     TOTAL_FLOORS = 10
210     goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
211     goal_block = Platform(0, goal_y, WIDTH, 40, GOLD, "goal")
212
213     for f in range(TOTAL_FLOORS):
214         floor_bottom = HEIGHT - (f * HEIGHT)
215         floor_top = HEIGHT - ((f + 1) * HEIGHT)
216
217         floor_plats = []
218         plat_y = floor_bottom - 150
219
220         while plat_y > floor_top + 50:
221             w = random.randint(60, 130)
222             x = random.randint(0, WIDTH - w)
223             floor_plats.append(Platform(x, plat_y, w, 20, COLOR_NORMAL, "normal"))
224             gap_y = random.randint(80, 130)
225             plat_y -= gap_y
226
227         if len(floor_plats) >= 2:
228             special_type = random.choice(["fake", "ice", "trampoline"])
229
230             # 10階の場合：ランダム特殊床1つ + トラップ床1つ
231             if f == TOTAL_FLOORS - 1:
232                 specials = random.sample(range(len(floor_plats)), 2)
233
234                 p_spec = floor_plats[specials[0]]
235                 # 種類に応じた色 (COLOR_MAP[special_type]) を適用
236                 floor_plats[specials[0]] = Platform(p_spec.get_rect().x, p_spec.get_rect().y, p_spec.get_rect().width, 20, COLOR_MAP[special_type])
237
238                 p_trap = floor_plats[specials[1]]
239                 # 赤色を適用
240                 floor_plats[specials[1]] = Platform(p_trap.get_rect().x, p_trap.get_rect().y, p_trap.get_rect().width, 20, COLOR_MAP["trap"])
241
242             # 1〜9階の場合：ランダム特殊床1つ
243             else:
244                 special_idx = random.choice(range(len(floor_plats)))
245                 p_spec = floor_plats[special_idx]
246                 floor_plats[special_idx] = Platform(p_spec.get_rect().x, p_spec.get_rect().y, p_spec.get_rect().width, 20, COLOR_MAP[special_type])
247
248         platforms.extend(floor_plats)
249

```

```
250 camera_y = 0
251 max_height = 0
252
253 running = True
254 while running:
255     for event in pygame.event.get():
256         if event.type == pygame.QUIT:
257             running = False
258
259     keys = pygame.key.get_pressed()
260
261     update_player(player, keys, platforms, goal_block)
262
263     p_rect = player.get_rect()
264
265     if player.get_is_reset():
266         camera_y = 0
267         player.set_is_reset(False)
268
269     SCROLL_TOP_MARGIN = 200
270     if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
271         camera_y = p_rect.y - SCROLL_TOP_MARGIN
272
273     SCROLL_BOTTOM_MARGIN = 150
274     camera_easing_down = 0.08
275     if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
276         target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
277         camera_y += (target_camera_y - camera_y) * camera_easing_down
278
279     if camera_y > 0:
280         camera_y = 0
281
282     current_height = (HEIGHT - 40) - p_rect.bottom
283     if current_height > max_height:
284         max_height = current_height
285
286     current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
287     display_floor = max(1, current_floor)
288
289     screen.fill(BLACK)
290
291     goal_rect = goal_block.get_rect()
292     goal_draw_y = goal_rect.y - camera_y
293     if -100 < goal_draw_y < HEIGHT + 100:
294         screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
295
296     for plat in platforms:
297         plat_rect = plat.get_rect()
298         draw_y = plat_rect.y - camera_y
299         if -50 < draw_y < HEIGHT + 50:
300             screen.blit(plat.get_image(), (plat_rect.x, draw_y))
301
302     player_draw_y = p_rect.y - camera_y
303     screen.blit(player.get_image(), (p_rect.x, player_draw_y))
304
305     if player.get_is_charging() and not player.get_is_clear():
306         gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
307         pygame.draw.rect(screen, COLOR_RED_UI, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
308
309     arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
310     pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
311
312     ui_text = font.render(f"Floor: {display_floor} / {TOTAL_FLOORS}    Max: {max_height // 10}m", True, WHITE)
313     screen.blit(ui_text, (10, 10))
314
315     if player.get_is_clear():
316         clear_text = large_font.render("CLEAR!!", True, GOLD)
```

```
317     text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
318     screen.blit(clear_text, text_rect)
319
320     sub_text = font.render("CONGRATULATIONS!", True, WHITE)
321     sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
322     screen.blit(sub_text, sub_rect)
323
324     pygame.display.flip()
325     clock.tick(FPS)
326
327     pygame.quit()
328     sys.exit()
329
330 if __name__ == "__main__":
331     main()
```